Neural Pre-Decoding for the Surface Code on Four FPGA Nodes: Integer Inference of Convolutional Pre-Decoders on Alveo U55C Accelerators

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing, Noida, 201307, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Neural pre-decoders remove local errors from the syndrome of a quantum error correcting code before a global decoder such as matching processes the remainder. We implement the two surface code pre-decoders that NVIDIA released with pretrained weights, four and six layer three dimensional convolutional networks, as integer networks on four AMD Alveo U55C FPGA cards, each card an independent node with two instances of one HLS kernel that runs both networks. The kernel uses 8 bit products, 32 bit accumulation and a table based GELU, keeps all activations on chip, and is bit exact with an integer model of the networks on every instance of every card. Measured with NVIDIA's evaluation code, the six layer network lowers the logical error rate of matching by a factor of 1.72 at physical error rate 0.006 and distance 13, as published, and the integer model by 1.48, while the four layer network gives little or no gain at 0.003. Four cards process 2,490 windows per second of the four layer network at distance 9 and 509 of the six layer network at distance 13, with linear scaling and within two percent of the kernel schedule, but only 1.2 and 1.6 times the same networks in PyTorch on a 256 thread host CPU. A single window takes 3.77 and 16.56 milliseconds, no faster than on the CPU. The arithmetic per window shows that real time pre-decoding at microsecond syndrome cycles requires networks designed for hardware, while the cards offload batch evaluation from the host.

Keywords: FPGA, Quantum error correction, Surface code, Neural decoder, Pre-decoder, Integer quantization, High-level synthesis, Multi-FPGA, Alveo U55C

1 Introduction

A fault tolerant quantum computer protects its logical qubits with an error correcting code and must decode the syndrome of that code while the computation runs [1, 2]. The rotated surface code [2, 3] is the leading candidate for superconducting hardware, where logical memories below threshold have now been demonstrated [4, 5]. Its standard decoder, minimum weight perfect matching, is implemented efficiently on CPUs by PyMatching and the Sparse Blossom algorithm [6, 7], and union find decoders trade some accuracy for a simpler and faster procedure [8]. Because a superconducting device produces a new round of syndrome data about every microsecond, a decoder that falls behind accumulates a backlog that eventually stalls the computation, and dedicated hardware such as FPGAs has been used to keep up [9–12]. Where syndrome cycles last milliseconds, as in trapped ion architectures, a single CPU can decode all logical qubits of a large application in real time [13]; the case for accelerators therefore rests on the microsecond budget of fast platforms.

Neural networks offer a second route. They have been trained as complete decoders [14–17] and, more recently, as *pre-decoders*: a local network removes the short error chains that dominate the syndrome and leaves a sparser residual syndrome to a global decoder. Chamberland et al. released a family of such pre-decoders, the NVIDIA Ising pre-decoders, as fully convolutional three dimensional networks over space and time, together with their training code and pretrained weights [18–21]. On GPUs the pre-decoder both lowers the logical error rate of uncorrelated matching and shortens the time PyMatching needs for the residual syndrome [18]. A convolutional network of fixed size, applied to a fixed window of syndrome data, is also a natural workload for an FPGA: its arithmetic is regular, its weights fit on chip, and integer arithmetic suffices for inference [22–25]. Overwater et al. analysed the hardware cost of neural decoders for this reason [26].

This paper places the two published surface code pre-decoders on four AMD Alveo U55C cards. Following our earlier multi FPGA work, each card is an independent compute node driven by its own host process [27]; a batch of syndrome windows is spread over the nodes, which is the shot parallel mode of a decoder serving many independent windows. The contributions are:

- An integer model of both networks (8 bit weights and activations, 32 bit accumulation, integer requantization and a 256 entry GELU table) that keeps NVIDIA's evaluation pipeline unchanged, so that the floating point networks, the integer model and the FPGA output are judged by the same logical error rate code.
- One HLS kernel that runs the four layer Fast network and the six layer Accurate network from the same bitstream, with all activations and partial sums on chip, and that is bit exact with the integer model in C simulation and on every instance of every card.
- Measurements on four cards: throughput on one, two and four nodes and latency per window, against the same networks in PyTorch on a 256 thread host CPU, and logical error rates of the floating point and integer networks against PyMatching alone.

- A reproduction of NVIDIA's published accuracy for both networks with their own evaluation code, which shows when a pre-decoder pays off: the small Fast network gives little or no gain at a physical error rate of $p = 0.003$, and the gains reported for the family come from the larger network and from higher noise.

Section 2 reviews decoders on FPGAs and neural decoders, Section 3 the pre-decoder and its evaluation, Section 4 the integer model, Section 5 the kernel and the node runtime, Section 6 the experimental setup, Section 7 the results and Section 8 their limits.

2 Related work

Decoders on FPGAs. LILLIPUT maps small surface codes to lookup tables in FPGA memory [9]. Liyanage et al. implemented a union find decoder for surface codes on an FPGA and later proposed a distributed union find decoder [10, 11]. Barber et al. demonstrated a real time, resource efficient decoder in FPGA and ASIC form [12]. These designs implement a complete decoder in custom logic; the present work instead accelerates a learned stage in front of an unchanged software decoder.

Neural decoders. Torlai and Melko trained a restricted Boltzmann machine as a decoder for topological codes [14]. Chamberland and Ronagh studied deep neural decoders for fault tolerance experiments and noted the latency constraints of real time use [15]. Gicev et al. proposed a convolutional decoder that scales to large distances [16], and AlphaQubit learned a high accuracy decoder from experimental data [17]. Overwater et al. explored the trade off between hardware cost and performance of neural network decoders for the surface code [26].

Pre-decoders. The NVIDIA Ising pre-decoders [18] are fully convolutional networks over a space and time window of the syndrome. They predict local data qubit corrections and syndrome flips; after the corrections are applied, PyMatching decodes the residual syndrome. Five surface code models of growing size are described; two are released with weights, the four layer model 1 ("Fast", receptive field 9) and the six layer model 4 ("Accurate", receptive field 13) [20, 21]. Their reference implementation targets GPUs and TensorRT, including reduced precision inference.

Neural networks on FPGAs. Weight stationary convolution engines with on chip buffers are the classic FPGA design for convolutional networks [23]; FINN and hls4ml generate quantized network accelerators from high level descriptions [24, 25], and integer only inference with per channel scales and requantization multipliers is the standard quantization scheme [22]. Our kernel follows these principles in hand written HLS [28, 29] so that its arithmetic matches an integer model that can be checked against NVIDIA's evaluation.

Multiple FPGAs as nodes. In our earlier work a quantum circuit statevector was distributed over four Alveo U55C cards, each card a node with its own host process and memory [27]. The same view applies here, without data exchange between cards because the windows are independent.

3 The Ising pre-decoders and their evaluation

Input window. The pre-decoder sees a window of T syndrome rounds of a rotated surface code of distance d as a tensor of shape $(4, T, d, d)$ [18]. Two channels carry the syndrome bits of the X and Z stabilizers, placed on the $d \times d$ grid of data qubits, and two channels carry fixed masks that mark which grid positions hold a stabilizer of each type. The syndrome channels take the values 0 and 1 and the mask channels the values 0, $\frac{1}{2}$ and 1.

Network. Both released surface code networks are stacks of three dimensional convolutions with $3 \times 3 \times 3$ kernels, zero padding that keeps the window size, and the tanh approximation of GELU [30] between layers, without normalization layers. The Fast network (model 1) has four layers, $4 \to 128 \to 128 \to 128 \to 4$ channels and 912,772 parameters; the Accurate network (model 4) has six layers, $4 \to 128 \to 128 \to 128 \to 128 \to 128 \to 4$ channels and about 1.8 million parameters [20, 21]. Their receptive fields are 9 and 13 sites, and they were trained on windows with $d = T = 9$ and $d = T = 13$. Because the networks are fully convolutional, the same weights apply to any larger window.

Output and use. The four output channels are logits of a correction on each data qubit (two channels) and of a flip of each syndrome bit (two channels). A logit at or above a threshold (zero in the released configuration) selects the correction. The corrections are applied to the syndrome, and PyMatching decodes the residual syndrome with the detector error model of the circuit; the logical outcome combines the pre-decoder corrections and the matching [7, 18].

Evaluation. NVIDIA's evaluation samples memory experiments with Stim [31] under a circuit level noise model with 25 parameters (state preparation, measurement, idling during CNOT layers and during preparation and reset, and the fifteen two qubit Pauli errors of a CNOT), which their public configuration sets for a physical error rate $p = 0.003$ [19]. For each shot it reports whether PyMatching alone and the pre-decoder followed by PyMatching recover the logical observable. We use this code without modification for every logical error rate in this paper, and add only the scaling of the 25 parameters to other values of p with the relations given in the public configuration (preparation and measurement $2p/3$, idling during CNOT layers $p/3$ per Pauli, idling during preparation and reset $2p/3 - 4p^2/9$ per Pauli, and $p/15$ per two qubit Pauli). We report the ratio of the number of logical errors of PyMatching alone to that of the pre-decoder followed by PyMatching on the same shots; a ratio above one is an improvement.

4 Integer model

The FPGA kernel computes an integer model of the network, defined in software first so that it can be judged by NVIDIA's evaluation and so that the kernel can be checked against it bit for bit.

Inputs. Multiplying the input tensor by two maps every entry to $\{0, 1, 2\}$, so the input is exact in integers with scale $\frac{1}{2}$; a site of the window needs two bits per channel, eight bits for its four channels.

Weights. The weights of each output channel are scaled to integers in $[-127, 127]$ with a per channel scale, the largest absolute weight of the channel divided by 127 [22], and the biases are rounded to 32 bit integers at the scale of the accumulator, so that a layer computes $a_o = b_o + \sum_{i,\tau} w_{oi\tau}\, x_i(\mathrm{site} + \tau)$ exactly in a 32 bit accumulator over the 27 kernel offsets τ and the input channels i.

Requantization and activation. For a hidden layer, the accumulator is mapped to an 8 bit index

$$q_o = \mathrm{clamp}\big(\lfloor (a_o M_o + 2^{23})/2^{24} \rfloor,\, -128,\, 127\big), \tag{1}$$

with a per channel integer multiplier M_o, and the activation of the next layer is $g[q_o]$, a 256 entry table of signed 8 bit values that holds the GELU of the dequantized index at the scale of the next layer. The multipliers and the table scales come from calibration: the 99.99th percentile of the absolute values of each layer's floating point pre-activations on syndrome windows of NVIDIA's inference pipeline sets the range of the index, and the same percentile of the activations sets the scale of the table output.

Decision. The last layer needs no requantization: a logit is at or above the threshold exactly when the accumulator is at or above the threshold divided by the accumulator scale, rounded up, so the kernel compares the 32 bit accumulator with an integer threshold per output channel and emits one bit.

Reference implementations. The integer model has a direct implementation in NumPy and a fast one that evaluates the convolutions in double precision, which represents every 32 bit accumulator exactly; the two agree bit for bit. The fast path supplies the network inside NVIDIA's evaluation module, in place of the floating point network, to obtain the logical error rate of the integer model.

5 Kernel and node runtime

5.1 Convolution engine

The kernel is written in C++ for Vitis HLS [29] and processes one window at a time through all layers of the network on a single convolution engine (Fig. 1). It stores the activations of a window on chip in two buffers that alternate between layers, each holding 128 channels of 8 bit values for up to $13 \times 13 \times 13$ sites, so that no intermediate result leaves the FPGA. The engine is weight stationary [23]: for a block of 64 output channels it keeps the 32 bit partial sums of every site on chip; for each of the 27 kernel offsets and each block of 16 input channels it loads a 64×16 block of 8 bit weights into registers and streams all sites of the window through the engine, one site per clock cycle. At each site the engine reads the 16 input activations of the neighbour at the kernel offset (zero outside the window, which realizes the zero padding of the network) and adds 1024 products to the 64 partial sums. After the last offset and input block, an epilogue pipeline adds the biases and either requantizes and maps the 64 results through the GELU table (hidden layers) or compares them with the thresholds and writes one bit per output channel (last layer). The weights of all layers stay in UltraRAM for the lifetime of the kernel and are loaded from HBM once per launch.

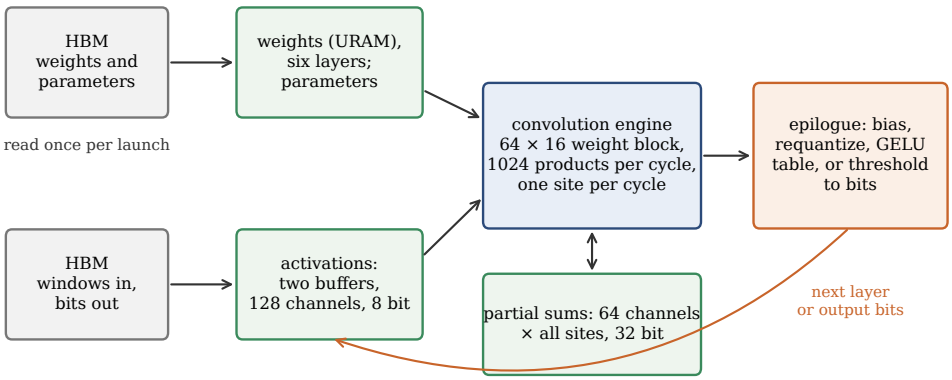


Fig. 1 Kernel of one instance. Weights and parameters are read from HBM into on chip memories once per launch. For each window, the convolution engine runs the layers one after the other; activations alternate between two on chip buffers, and the partial sums of 64 output channels stay on chip for all sites. Each card holds two instances, in SLR1 and SLR2.

5.2 Epilogue parameters

A first version read the bias, multiplier and table entry of each of the 64 output channels per cycle from one parameter memory, at addresses that depend on the layer. HLS partitioned that memory into 144 banks and had not finished scheduling the epilogue when it was stopped after 7,000 seconds. The final kernel copies each layer's biases, multipliers and thresholds into registers and its activation table into 64 small per channel memories before the sites stream through, so the epilogue reads every parameter at a fixed lane. With this change C synthesis finishes in about two minutes, every loop of the engine and epilogue reaches an initiation interval of one, and the estimated maximum clock frequency is 370 MHz against the 300 MHz target.

5.3 One bitstream for both networks

Both networks have 128 hidden channels and differ only in the number of layers, so the layer count is a launch argument (four or six) and the on chip weight and parameter memories are sized for six layers. The host derives the layer count from the size of the parameter file; the kernel reads only the weights and parameters of the layers the host supplies. One bitstream therefore runs the Fast network and the Accurate network.

5.4 Throughput model

The work per window is fixed by the network and the window size: 665,127,936 multiply accumulate operations for the Fast network at $d = T = 9$ and 3,948,272,640 for the Accurate network at $d = T = 13$. From the loop structure and the pipeline depths reported by HLS, a window takes 935,416 cycles (Fast, $d = 9$) or 4,613,082 cycles (Accurate, $d = 13$) on one instance, that is 711 and 856 useful operations per cycle out of the 1024 the engine can perform. The difference to the peak comes from

the first layer, which has four input channels on an engine built for 16, the last layer, which has four output channels on an engine built for 64, and the pipeline fill at each kernel offset. At 300 MHz the model predicts 319 windows per second per instance for the Fast network at $d = 9$ and 65 for the Accurate network at $d = 13$.

5.5 Cards as nodes

Each card carries two kernel instances, placed in SLR1 and SLR2 because SLR0 also holds the platform's HBM and DMA logic; each instance has its own two HBM pseudo channels, one for weights and parameters and one for windows and output bits. As in our multi FPGA statevector work [27], every card is a node driven by its own host process through XRT [32]. A process loads the bitstream, copies the weights and parameters to both instances once, and then keeps two launches per instance in flight so that the transfer of one batch of windows overlaps the computation of the previous one. For multi card runs all processes finish their setup, wait at a common start barrier, and report the wall clock interval of their timed loop; the aggregate throughput is the total number of windows divided by the interval from the first start to the last end. The windows of a batch are independent, so the nodes exchange no data.

6 Experimental setup

Hardware. The host has two AMD EPYC 7742 processors with 256 hardware threads in total and 504 GiB of memory, and four AMD Alveo U55C cards [33] on PCIe, each with 16 GiB of HBM2 and the shell `xilinx_u55c_gen3x16_xdma_base_3`, driven by XRT 2.16.204 [32]. The kernel was synthesized with Vitis HLS 2023.2 and linked with Vitis 2023.2 at a 300 MHz kernel clock target, with the two instances constrained to SLR1 and SLR2.

Networks and evaluation. The pretrained Fast (checkpoint v1.0.77) and Accurate (v1.0.86) networks were downloaded from NVIDIA's model pages and checked against their published SHA256 hashes [20, 21]. Logical error rates were computed with NVIDIA's evaluation code at commit 33acb15 [19], run on the host CPU with PyTorch 2.14.1, Stim 1.16.0 [31] and PyMatching 2.4.0 [7], for memory experiments in both the X and the Z basis with $T = d$ rounds. The integer model was calibrated on 2,048 windows (Fast) or 1,024 windows (Accurate), half from each basis, drawn from the same pipeline at the same noise level, separate from the evaluation shots.

Card measurements. Each card measurement starts with a bit exact check: a batch of windows from the inference pipeline (eight test windows, after a calibration of the integer model on 256 other windows) runs on both instances of every card, and the output bits are compared with the integer model. Throughput is then measured on one, two and four cards with batches of 64 windows per launch, two launches in flight per instance, and about 20 seconds per point, three repetitions per point; we report the median. Latency is the time of one launch of a single window with one launch in flight, averaged over 200 launches; it includes the host to card transfer of the window and the transfer of the result back. The card temperatures are read before and after every point. A logical error rate evaluation ran on the host CPU during the

Table 1 Logical errors over both bases (X and Z, each with the stated number of shots) of PyMatching alone and of the pre-decoder followed by PyMatching on the same shots, with $T = d$ rounds. Ratio, X: improvement factor in the X basis (PyMatching errors divided by errors with the pre-decoder); NVIDIA, X: the published factor for the same network, p and d (Tables 4 and 5 of [18]).

Network	Arithmetic	d	p	Shots per basis	PyMatching	With pre-decoder	Ratio, X	NVIDIA, X
Fast	FP32	9	0.003	1,048,576	2,605	2,479	1.08	1.10
Fast	INT8	9	0.003	65,536	154	154	0.80	1.10
Fast	FP32	13	0.003	1,048,576	519	586	0.90	0.91
Fast	FP32	9	0.006	1,048,576	61,392	51,883	1.17	1.24
Fast	INT8	9	0.006	65,536	3,887	3,317	1.14	1.24
Accurate	FP32	13	0.003	524,288	223	176	1.29	1.32
Accurate	FP32	13	0.006	262,144	11,577	6,747	1.72	1.76
Accurate	INT8	13	0.006	65,536	2,849	1,922	1.51	1.76

card measurements; the host processes of the cards only start launches and copy data, and the throughput agrees with the cycle model of the kernel (Section 7.2).

CPU baseline. The same floating point networks run in PyTorch 2.14.1 on the host, in inference mode, with the cards idle and no other job running, with 32, 64, 128 and 256 threads and batch sizes from 1 to 256, each point for 10 seconds and repeated 2 times. We report the best throughput over threads, batch sizes and repetitions, and the best throughput at batch size one; on this two socket host fewer threads are often faster than all of them. Reduced precision (bfloat16) was much slower than single precision on this processor and is not used.

7 Results

7.1 Logical error rates

Table 1 lists the logical errors of PyMatching alone and of the pre-decoder followed by PyMatching on the same shots, for both networks, both noise levels and both arithmetics, together with the improvement factors NVIDIA published for the X basis [18]; Fig. 2 compares the X basis factors.

Agreement with the published values. With NVIDIA's own evaluation code our measurements follow the published factors in the X basis. The Fast network at $p = 0.003$ gives 1.08 at $d = 9$ and 0.90 at $d = 13$ (1,048,576 shots per basis each), against 1.10 and 0.91 published, and at $p = 0.006$, $d = 9$ 1.17 (1,048,576 shots per basis), against 1.24. The Accurate network at $d = 13$ gives 1.29 at $p = 0.003$ (524,288 shots per basis) and 1.72 at $p = 0.006$ (262,144 shots per basis), against 1.32 and 1.76. Every factor we measure is within 0.07 of the published one and slightly below it.

When the pre-decoder pays off. The small Fast network gives at best a modest gain: at $p = 0.003$ it lowers the logical error rate by a factor of only 1.05 over both bases at $d = 9$ and raises it at $d = 13$ (0.89), in agreement with the factor below one that NVIDIA reports for this network at this noise level. The Accurate network, with two more layers and a receptive field of 13, lowers the logical error rate by a factor

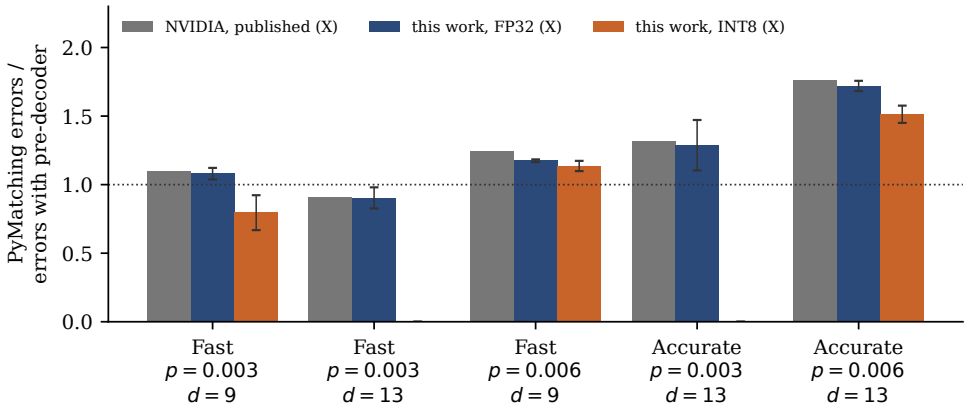


Fig. 2 Improvement factor of the pre-decoder over PyMatching alone in the X basis: NVIDIA's published values and the values measured here with NVIDIA's evaluation code, for the floating point (FP32) and the integer (INT8) networks. Error bars: one standard deviation from the counts of logical errors. Values below the dotted line mean that the pre-decoder increases the logical error rate.

of 1.72 over both bases at $p = 0.006$ and by 1.27 at $p = 0.003$, both at $d = 13$. The gains quoted for the network family therefore depend on the network size and the noise level, and a hardware implementation should target the larger network.

Integer arithmetic. The integer model of the Fast network gives a factor of 1.17 over both bases at $p = 0.006$, $d = 9$ (65,536 shots per basis), against 1.18 in floating point, and the integer Accurate network gives 1.48 at $p = 0.006$, $d = 13$ (65,536 shots per basis), against 1.72. The integer model thus keeps the gain of the four layer network but loses about a third of the gain of the six layer network, plausibly because the rounding of the activations accumulates over five hidden layers; with 1,922 errors against 1,660 expected on the same shots from the floating point factor, the loss is well outside the counting uncertainty. At $p = 0.003$, $d = 9$ the integer Fast network gives 1.00 over both bases, with too few errors (65,536 shots per basis) to distinguish it from the floating point network. Since the kernel output is bit exact with the integer model (Section 7.2), these are the logical error rates the cards deliver.

7.2 Hardware

Implementation. After routing and post route physical optimization, the two instance design misses the 300 MHz target by a worst negative slack of -0.013 ns, and Vitis lowers the kernel clock to 298 MHz. Table 2 lists the routed resources of one instance. The routed design uses more DSP blocks and BRAM and fewer URAM blocks than the HLS estimate, because the implementation maps part of the products to DSP blocks and part of the on chip memories to BRAM; DSP blocks are the largest share, at less than half of one SLR.

Bit exactness. All 8 checks (two networks on four cards) passed: the output bits of all 16 instance runs equal those of the integer model for every site and output channel of the test windows. Since the kernel computes the integer model exactly for

Table 2 Routed resources of one kernel instance (the second instance is within 2% of these values), as reported by Vivado after implementation, and their share of one SLR of the U55C.

Resource	One instance	Percent of one SLR
LUT	38,496	8.9
Flip flops	50,553	5.8
DSP blocks	1,296	43.1
BRAM (36 kbit)	269	40.0
URAM	114	35.6

Table 3 Throughput in windows per second on one, two and four cards (two instances per card), median and range of three repetitions of about 20 seconds each. Scaling: throughput divided by the number of cards times the throughput of one card.

Network	Cards	Windows/s (median)	Range	Per card	Scaling
Fast, $d = 9$	1	623	622 to 623	623	1.00
Fast, $d = 9$	2	1,245	1,245 to 1,245	622	1.00
Fast, $d = 9$	4	2,490	2,490 to 2,490	622	1.00
Accurate, $d = 13$	1	127	127 to 127	127	1.00
Accurate, $d = 13$	2	255	255 to 255	127	1.00
Accurate, $d = 13$	4	509	509 to 509	127	1.00

any calibration, the logical error rates of the integer model in Table 1 are those of the cards.

Throughput. Table 3 and Fig. 3 give the measured throughput. One card processes 623 windows per second of the Fast network at $d = 9$ and 127 of the Accurate network at $d = 13$, and four cards 2,490 and 509: the scaling from one to four cards is 1.00 and 1.00, as expected for nodes that exchange no data, and the three repetitions of each point agree to within one window per second. One instance reaches 98% (Fast) and 99% (Accurate) of the throughput predicted by the cycle model at 298 MHz, so the transfers of the windows and of the output bits are hidden behind the computation and the kernel runs at the rate its schedule allows. In products per second, four cards perform 1.66×10^{12} (Fast) and 2.01×10^{12} (Accurate) useful 8 bit multiply accumulate operations per second. During the measurements the card temperatures stayed at or below 83 °C.

Comparison with the host CPU. PyTorch on the host reaches at best 2,038 windows per second for the Fast network (64 threads, batch size 16) and 318 for the Accurate network (256 threads, batch size 16); at batch size one it reaches 257 and 114, that is 3.89 ms and 8.77 ms per window. Four cards are therefore only 1.2 times (Fast) and 1.6 times (Accurate) faster than the host CPU at its best setting, and one card is slower than the host. The throughput of the CPU varies between the two repetitions

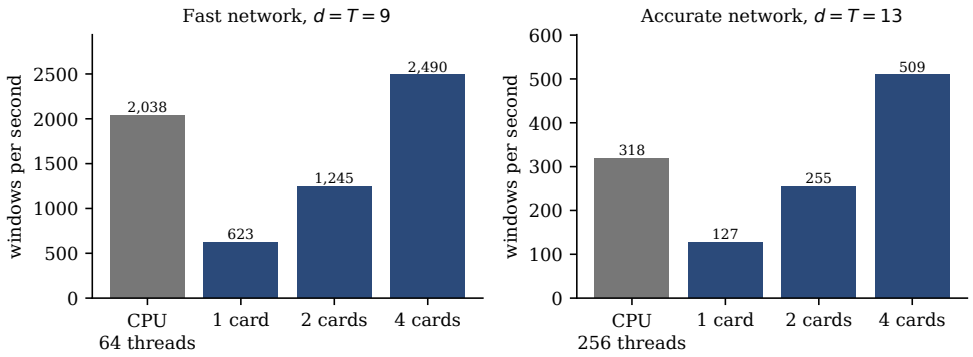


Fig. 3 Throughput of the floating point networks in PyTorch on the host CPU (best thread count and batch size) and of the integer networks on one, two and four cards.

of a point, by a median of 12% (Fast) and 4% (Accurate) and by up to 108% at single points; the comparison uses the best repetition, which favours the CPU.

Latency. With one window per launch and one launch in flight, a window takes 3.77 ms (Fast, $d = 9$) and 16.56 ms (Accurate, $d = 13$) from the start of the transfer to the card until its bits are back on the host. The computation alone takes 3.14 ms and 15.48 ms according to the cycle model; the remaining 0.63 ms and 1.08 ms cover the transfers, the kernel launch through XRT and the loading of the weights from HBM, which each launch repeats.

8 Discussion

Distance to real time decoding. A superconducting device produces a syndrome round about every microsecond [5]. To keep up with one logical qubit, a pre-decoder that consumes windows of $T = d$ rounds must finish at least one window every d microseconds: 111,111 windows per second for the Fast network at $d = 9$ and 76,923 for the Accurate network at $d = 13$. Because the work per window is fixed, this asks for $74{\times}10^{12}$ and $304{\times}10^{12}$ multiply accumulate operations per second. Four cards deliver $1.66{\times}10^{12}$ and $2.01{\times}10^{12}$, and even if every one of the 9,024 DSP blocks of a U55C [33] performed two 8 bit products per cycle at 300 MHz, one card would reach $5.4{\times}10^{12}$. The released networks evaluated densely on full windows therefore need more than ten times the peak arithmetic of a card of this class for real time use, independently of how well the engine is scheduled; our kernel already reaches 98% and 99% of the rate its schedule allows. The measured latency of 3.77 ms per window leads to the same conclusion. Real time pre-decoding in hardware therefore needs networks designed for it: fewer channels, fewer layers, or sliding windows that reuse the results of earlier rounds instead of recomputing a full window, and possibly the sparsity of the syndrome, which in our kernel only the first layer could exploit because the hidden activations after GELU are dense. On platforms with millisecond syndrome cycles, a CPU already decodes in real time [13], and a pre-decoder is not needed for speed.

What the cards are useful for. The design is a throughput engine. Evaluating a pre-decoder on many shots, as in Table 1, and generating pre-decoded syndromes

for decoder studies are batch tasks of independent windows, which spread over the cards without communication and scale linearly with their number. For these tasks, however, the advantage of the cards over the host is small: four cards are 1.2 (Fast) and 1.6 (Accurate) times faster than PyTorch on the two socket, 128 core host at its best thread count and batch size, so the host matches 3.3 cards for the Fast network and 2.5 for the Accurate network, and for a single window the CPU is as fast (Fast) or faster (Accurate). What the cards offer is that the work leaves the host, whose cores then remain free for the matching decoder, and an integer output known to be bit exact. The CPU baseline is the floating point network in PyTorch, the form in which the networks are released; an optimized integer CPU implementation could close or reverse the remaining gap, and we did not measure GPUs or the power of either platform. The per card throughput is limited by the single engine of 1,024 products per cycle per instance, which uses less than half of the DSP blocks of an SLR (Table 2); a larger engine or two products per DSP block would raise it, but not by the factor that real time use needs.

Accuracy of the integer model. The integer model keeps the gain of the Fast network and about two thirds of that of the Accurate network: the Fast network at $p = 0.006$, $d = 9$ gives 1.17 against 1.18, and the Accurate network at $p = 0.006$, $d = 13$ gives 1.48 against 1.72, both over both bases. Calibration uses windows of both bases at the noise level of the evaluation and no retraining; quantization aware training could recover the remainder.

Limits of this study. The kernel holds windows up to $13 \times 13 \times 13$ sites and the two released surface code networks; larger windows need tiling of the activation buffers. Every launch reloads the weights from HBM, which costs little in throughput but adds to the latency of a single window; a kernel that keeps the weights resident between launches would remove that part. Logical error rates are measured for memory experiments under NVIDIA's circuit level noise model at two noise levels, with the number of shots given in Table 1; the smallest counts, at $p = 0.003$, carry a relative statistical uncertainty of up to about ten percent.

9 Conclusions

We implemented the two released NVIDIA Ising surface code pre-decoders as one integer HLS kernel on four Alveo U55C cards, each card a node with two kernel instances. The kernel is bit exact with an integer model of the networks on every instance of every card, and the integer model is evaluated with NVIDIA's unmodified evaluation code. With that code we reproduce the published accuracy of both networks: the Accurate network lowers the logical error rate of PyMatching by a factor of 1.72 at $p = 0.006$, $d = 13$, while the Fast network gives little or no gain at $p = 0.003$. Four cards process 2,490 windows per second of the Fast network at $d = 9$ and 509 of the Accurate network at $d = 13$, with linear scaling over the cards and 98% and 99% of the throughput predicted by the kernel schedule, but only 1.2 and 1.6 times the throughput of the networks in PyTorch on the 256 thread host. The latency of a single window, 3.77 ms and 16.56 ms, and the arithmetic the networks need per window show that real time pre-decoding at microsecond syndrome cycles requires networks designed for hardware

rather than faster engines for the released ones. Smaller and incremental networks, larger engines, resident weights and quantization aware training are the next steps.

Acknowledgments. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India. The research was conducted at the C-DAC Noida Centre under the supervision of Mr. Abhishek Tiwari, Scientist F, Embedded Systems Division, C-DAC Noida. We thank NVIDIA for releasing the Ising pre-decoders, their training and evaluation code and their pretrained weights, and the developers of Stim and PyMatching, on which the evaluation of this work depends.

Declarations

Funding. This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India.

Conflict of interest. The author declares no competing interests.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. The raw measurement records that support the findings of this study (one JSON file per run with timestamps, bitstream hash, card identifiers and temperatures), the logical error rate records and the scripts that generate every table and figure from them are available from the corresponding author on reasonable request. The pretrained networks are available from NVIDIA under the NVIDIA Open Model License [20, 21].

Author contribution. N. Ali designed and implemented the integer model, the HLS kernel, the host runtime and the evaluation harness, carried out the measurements and wrote the manuscript.

References

[1] Dennis, E., Kitaev, A., Landahl, A., Preskill, J.: Topological quantum memory. Journal of Mathematical Physics **43**(9), 4452–4505 (2002) https://doi.org/10.1063/1.1499754

[2] Fowler, A.G., Mariantoni, M., Martinis, J.M., Cleland, A.N.: Surface codes: Towards practical large-scale quantum computation. Physical Review A **86**(3), 032324 (2012) https://doi.org/10.1103/physreva.86.032324

[3] Kitaev, A.Y.: Fault-tolerant quantum computation by anyons. Annals of Physics **303**(1), 2–30 (2003) https://doi.org/10.1016/s0003-4916(02)00018-0

[4] Google Quantum AI: Suppressing quantum errors by scaling a surface code logical qubit. Nature **614**(7949), 676–681 (2023) https://doi.org/10.1038/s41586-022-05434-1

[5] Google Quantum AI and Collaborators: Quantum error correction below the surface code threshold. Nature **638**(8052), 920–926 (2025) https://doi.org/10.1038/s41586-024-08449-y

[6] Higgott, O.: Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching. ACM Transactions on Quantum Computing **3**(3), 1–16 (2022) https://doi.org/10.1145/3505637

[7] Higgott, O., Gidney, C.: Sparse blossom: correcting a million errors per core second with minimum-weight matching. Quantum **9**, 1600 (2025) https://doi.org/10.22331/q-2025-01-20-1600

[8] Delfosse, N., Nickerson, N.H.: Almost-linear time decoding algorithm for topological codes. Quantum **5**, 595 (2021) https://doi.org/10.22331/q-2021-12-02-595

[9] Das, P., Locharla, A., Jones, C.: Lilliput: a lightweight low-latency lookup-table decoder for near-term quantum error correction. In: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 541–553 (2022). https://doi.org/10.1145/3503222.3507707

[10] Liyanage, N., Wu, Y., Deters, A., Zhong, L.: Scalable quantum error correction for surface codes using fpga. In: 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 916–927 (2023). https://doi.org/10.1109/qce57702.2023.00106

[11] Liyanage, N., Wu, Y., Tagare, S., Zhong, L.: Fpga-based distributed union-find decoder for surface codes. IEEE Transactions on Quantum Engineering **5**, 1–18 (2024) https://doi.org/10.1109/tqe.2024.3467271

[12] Barber, B., Barnes, K.M., Bialas, T., Buğdaycı, O., Campbell, E.T., Gillespie, N.I., Johar, K., Rajan, R., Richardson, A.W., Skoric, L., Topal, C., Turner, M.L., Ziad, A.B.: A real-time, scalable, fast and resource-efficient decoder for a quantum computer. Nature Electronics **8**(1), 84–91 (2025) https://doi.org/10.1038/s41928-024-01319-5

[13] Ye, M., Maksymov, A., Delfosse, N.: Real-time decoder for a megaquop quantum computer using a single cpu. arXiv preprint arXiv:2608.25027 (2026) arXiv:2608.25027

[14] Torlai, G., Melko, R.G.: Neural decoder for topological codes. Physical Review Letters **119**(3), 030501 (2017) https://doi.org/10.1103/physrevlett.119.030501

[15] Chamberland, C., Ronagh, P.: Deep neural decoders for near term fault-tolerant experiments. Quantum Science and Technology **3**(4), 044002 (2018) https://doi.org/10.1088/2058-9565/aad1f7

[16] Gicev, S., Hollenberg, L.C.L., Usman, M.: A scalable and fast artificial neural network syndrome decoder for surface codes. Quantum **7**, 1058 (2023) https://doi.org/10.22331/q-2023-07-12-1058

[17] Bausch, J., Senior, A.W., Heras, F.J.H., Edlich, T., Davies, A., Newman, M., Jones, C., Satzinger, K., Niu, M.Y., Blackwell, S., Holland, G., Kafri, D., Atalaya, J., Gidney, C., Hassabis, D., Boixo, S., Neven, H., Kohli, P.: Learning high-accuracy error decoding for quantum processors. Nature **635**(8040), 834–840 (2024) https://doi.org/10.1038/s41586-024-08148-8

[18] Chamberland, C., Olle, J., Li, M., Thornton, S., Baratta, I.: Fast and accurate ai-based pre-decoders for surface codes. arXiv preprint arXiv:2604.12841 (2026) arXiv:2604.12841

[19] NVIDIA: Ising-Decoding: open source training and inference code of the Ising pre-decoders, commit 33acb15. https://github.com/NVIDIA/Ising-Decoding (2026)

[20] NVIDIA: Ising-Decoder-SurfaceCode-1-Fast, model card, checkpoint v1.0.77. https://huggingface.co/nvidia/Ising-Decoder-SurfaceCode-1-Fast (2026)

[21] NVIDIA: Ising-Decoder-SurfaceCode-1-Accurate, model card, checkpoint v1.0.86. https://huggingface.co/nvidia/Ising-Decoder-SurfaceCode-1-Accurate (2026)

[22] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2704–2713 (2018). https://doi.org/10.1109/cvpr.2018.00286

[23] Zhang, C., Li, P., Sun, G., Guan, Y., Xiao, B., Cong, J.: Optimizing fpga-based accelerator design for deep convolutional neural networks. In: Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 161–170 (2015). https://doi.org/10.1145/2684746.2689060

[24] Umuroglu, Y., Fraser, N.J., Gambardella, G., Blott, M., Leong, P., Jahre, M., Vissers, K.: Finn. In: Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 65–74 (2017). https://doi.org/10.1145/3020078.3021744

[25] Duarte, J., Han, S., Harris, P., Jindariani, S., Kreinar, E., Kreis, B., Ngadiuba, J., Pierini, M., Rivera, R., Tran, N., Wu, Z.: Fast inference of deep neural networks in fpgas for particle physics. Journal of Instrumentation **13**(07), 07027–07027 (2018) https://doi.org/10.1088/1748-0221/13/07/p07027

[26] Overwater, R.W.J., Babaie, M., Sebastiano, F.: Neural-network decoders for quantum error correction using surface codes: A space exploration of the hardware cost-performance tradeoffs. IEEE Transactions on Quantum Engineering **3**, 1–19 (2022) https://doi.org/10.1109/tqe.2022.3174017

[27] Ali, N.: Multi-FPGA Quantum Circuit Simulation: A Distributed Statevector

Architecture on Four Xilinx Alveo U55C Accelerators. Research Square preprint. Under major revision, The Journal of Supercomputing (2026). https://doi.org/10.21203/rs.3.rs-10128153/v1

[28] Cong, J., Liu, B., Neuendorffer, S., Noguera, J., Vissers, K., Zhang, Z.: High-level synthesis for fpgas: From prototyping to deployment. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **30**(4), 473–491 (2011) https://doi.org/10.1109/tcad.2011.2110592

[29] Advanced Micro Devices: Vitis High-Level Synthesis User Guide (UG1399), version 2023.2. https://docs.amd.com/r/en-US/ug1399-vitis-hls (2023)

[30] Hendrycks, D., Gimpel, K.: Gaussian error linear units (gelus). arXiv preprint arXiv:1606.08415 (2016) arXiv:1606.08415

[31] Gidney, C.: Stim: a fast stabilizer circuit simulator. Quantum **5**, 497 (2021) https://doi.org/10.22331/q-2021-07-06-497

[32] Advanced Micro Devices: Xilinx Runtime (XRT) Documentation. https://xilinx.github.io/XRT/ (2023)

[33] Advanced Micro Devices: Alveo U55C Data Center Accelerator Card Data Sheet (DS978). https://docs.amd.com/r/en-US/ds978-u55c (2023)